Quantum Computing Meets Rust

Harnessing Memory Safety and Concurrency for Next-Generation Frameworks

Kethan Pilla
Computer Science
University of North Alabama
Florence Alabama USA
kethanpilla11@gmail.com

I. Introduction

Quantum computing has become a game-changing technology with its influence on the wide range of scientific and industrial fields in processing power and problem-solving capacity. However, quantum computational frameworks are so complicated that they require exact concurrent handling, strict software safety, and effective memory management—areas in which many traditional programming languages are inadequate. Rust stands as a strong rival for resolving these affairs. Rust is famous for its strong concurrency architecture and rigorous memory.

This study explores the possibility of using Rust's built-in concurrency and safety characteristics to create frameworks for next-generation quantum computing. Our primary research question is: "How can Rust's strong memory-safety guarantees and concurrency model be leveraged to develop next-generation quantum computing frameworks?" By handling parallel computing tasks and averting memory related errors, Rust has made developments in the superior quantum software solutions, this paper brings out the practical outcomes of blending Rust into quantum computing contexts.

II. Background

1.1 Historical Background

Rust is an open-source systems programming language developed by Graydon Hoare as a personal project in 2006. The project gained momentum when Mozilla adopted it in 2009, aiming to address memory safety issues prevalent in traditional systems programming languages such as C and C++. Without sacrificing safety and possessing the features of strong memory safety guarantees, and efficient performance, Rust was officially launched as version 1.0 in 2015. Designed with the goal of eliminating common memory errors like leaks, double-free errors, and segmentation faults, Rust quickly became popular in areas requiring robust safety guarantees such as embedded systems, web browsers, and performance-critical applications [1].

1.2 Language Overview

Rust is a statically typed multi-paradigm programming language with imperative and object-oriented programming features. It provides memory safety without needing a garbage collector or a runtime overhead, instead relying on concepts such as ownership, borrowing, and lifetimes. Ownership ensures memory safety by guaranteeing that each value has a single owner, with clearly defined rules for accessing memory, significantly reducing common memory-related errors. [2] [1].

The syntax of Rust is highly influenced by C and C++, making it accessible to system programmers, while its semantics ensure rigorous compile-time checking to enforce correctness. Rust supports pattern matching, generics, type inference, and algebraic data types (ADTs), enabling developers to build highly abstract yet performant systems. Control structures in Rust include conditional statements and loops.

Additionally, Rust supports concurrency natively through the concept of "fearless concurrency." This allows developers to write concurrent programs without the usual risks associated with thread synchronization issues such as race conditions or deadlocks. Rust enforces thread safety at compile-time through its type system, specifically ownership rules and borrowing, ensuring safe concurrent execution on multi-core processors without the traditional complexity or performance penalties associated with explicit locking mechanisms. [3].

1.3 Fearless Concurrency

Concurrency is the management of multiple computations through threads. It influences multi-core CPU architecture and helps enhance performance. Safety is ensured through strict compile-time checks. Explicit synchronization mechanisms are important for Threads in Rust to share a mutable state. This can be enforced by Rust's ownership rules, making concurrent programs easy to reason and inherent safe. [2].

Rust distinguishes between concurrency and parallelism. With the built-in concurrency primitives of Rust, developers are benefitted from reliable concurrent applications and minimized risks.

1.4 Memory Safety and Guarantees

1.4.1 Programs and Memory. Programs, which are executed by an operating system, run as processes and are allocated with virtual address space. It ensures critical security and stability by isolation between processes. A program needs memory to store instructions, manage runtime resources, and keep track of execution flow, including function calls, local variables, and control flow returns.

1.4.2 Accessing Memory. Processes use virtual memory managed by the operating system through structures called page tables to interact with physical memory. When memory is requested, the OS provides virtual addresses, internally mapped to

physical RAM locations. Memory management involves two primary actions: allocation, where memory is provided to a process, and deallocation, where memory is returned for reuse.

In languages like C and C++, manual memory management was prevalent historically. But this manual management is usually prone to error and can result in issues like leakage of memory and faults in segmentation. These challenges are circumvented by Rust by automating memory management by compile-time checks of matters of ownership, and it also reduces the need for manual management and garbage collection and further ensures guarded and efficient memory usage. [3] [1].

1.5 Quantum Computing Frameworks and Rust's Advantage.

Quantum computing facilitates the simulation and execution of quantum algorithms and gives away unique challenges due to its nature, and complexity of state space and stringent performance. Guarantees of quantum computing applications need attentive safety, efficient memory management and concurrency handling.

We can reduce bugs and memory errors that occur in low-level programming with the help of Rust's intrinsic memory safety mechanisms, which is essential for quantum algorithm accuracy and stability. Rust's concurrency model allows effective use of multicore CPUs, essential for simulating quantum circuits efficiently. Rust-based quantum emulators, such as state-vector emulators, have demonstrated performance efficiency comparable to traditional C/C++ implementations but with improved safety and maintainability. We are provided various benefits over competitors through Rust's rigorous compile-time checks and efficient memory use and it also stimulates shielded and highly performant quantum computing software by making Rust an eminent product for the next generation of quantum frameworks.

III. Literature Review

2.1 Introduction.

Quantum computing frameworks comprise various circuit simulators, emulators, and system libraries that emphasize high performance and reliability. These frameworks involve complex memory operations and parallel computations. Historically, low-level languages like C/C++ have been used for performance, at the cost of potential memory errors, while high-level languages offer safety but often lack speed. Rust has emerged as a systems programming language promising a rare combination: strong memory safety guarantees and a modern concurrency model without sacrificing performance. [4]. Rust "provides both memory and thread safety like Java, and runtime efficiency like C/C++, by introducing a number of novel features such as ownership, borrowing, explicit lifetime, and unsafe" [5]. In theory, these features could make Rust an ideal foundation for developing next-generation quantum computing frameworks that are both safe (free from memory bugs and data races) and fast (able to utilize multi-core CPUs and GPUs).

This survey is about the developments and recent research at the intersection of Rust and quantum computing frameworks. The beginning can be stated with the relevance of Rust to scientific computing and background on Rust's ownership-based memory safety model. The main argument put forth is that Rust's unique design can drastically improve the reliability and performance of quantum software. We can assume that developers can create quantum simulators and libraries that use traditional C or C++ implementations which eliminate certain classes of bugs by leveraging Rust's timely safety checks and fearless concurrency model. To explore this, we organize the discussion into thematic sections:

(1) Memory Safety in Rust,
(2) Fearless Concurrency for Parallel Quantum Computing,
(3) Rust in Quantum Simulation and Emulation, and
(4) Safe Use of "Unsafe" Rust for High-Performance Needs.

Each theme compares viewpoints from multiple sources, highlighting agreements, disagreements, and unique insights. We also address cross-cutting concerns like performance optimizations (e.g. zero-cost abstractions, GPU integration) and the growing ecosystem of Rust in scientific computing.

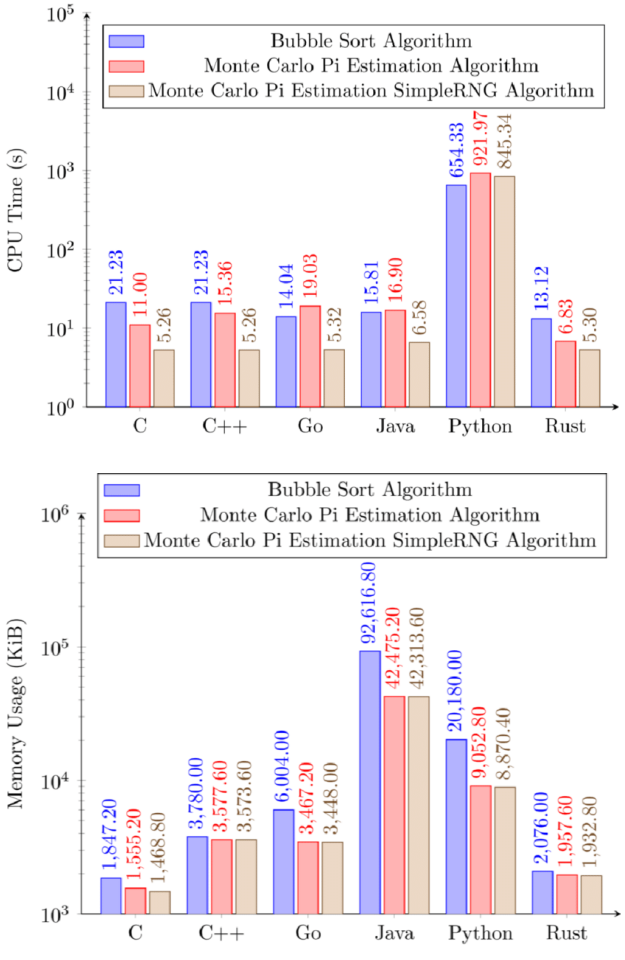


Figure 1: CPU time comparison of Bubble Sort, Monte Carlo Pi estimation, and Monte Carlo Pi estimation with SimpleRNG

algorithms across multiple programming languages. The results highlight Rust's efficient computational performance relative to other languages. Adapted from "The Programming Language for Safety and Performance" [4].

Figure 2: Memory usage comparison of Bubble Sort, Monte Carlo Pi estimation, and Monte Carlo Pi estimation with SimpleRNG algorithms across different programming languages, emphasizing Rust's efficient memory management capabilities. Adapted from "The Programming Language for Safety and Performance" [4].

2.2 Memory Safety as a Pillar for Quantum Computing Frameworks.

Memory management bugs are notoriously common in low-level languages and can be catastrophic in quantum computing applications where correctness is paramount. Rust was designed to tackle this problem at the root by enforcing memory safety *at compile time*. Unlike C/C++, which "are infamous for a lot of memory vulnerability reports in software," Rust offers a "better, compile-time solution to memory management" [1]. Its ownership and borrowing system ensure that all references point to valid memory and that resources are freed without use-after-free or double-free errors. In practice, this means Rust code is much less prone to buffer overflows or dangling pointers, issues that could corrupt quantum simulation data structures. As one textbook observes, Rust "makes it hard to write software that leaks memory" and actively *discourages bad programming practices*, guiding developers toward code that "uses memory safely and efficiently." [1]. This built-in safety is especially valuable for long-running quantum simulations that allocate large state vectors or matrices – memory errors that might only surface after hours of computation are prevented by Rust's compiler upfront.

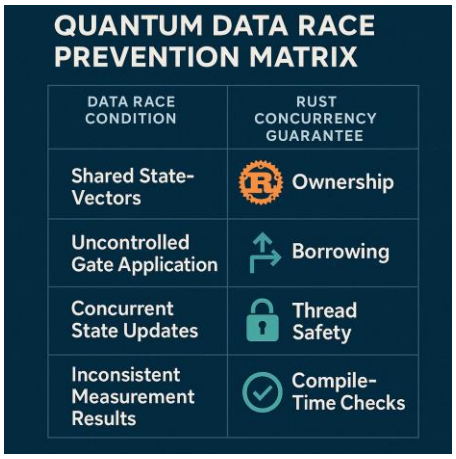
Several studies confirm Rust's effectiveness at ensuring memory safety without introducing performance overhead. Bugden and Alahmar's survey of Rust notes that Rust achieves memory safety "without requiring the use of a garbage collector" and that it is *"outstanding for enforcing memory safety"*, ensuring all pointers are valid at runtime. [4]. Importantly, Rust attains this safety while adopting C++'s zero-cost abstraction principle – high-level constructs cost nothing at runtime. Rust's focus on safety did not come at the expense of speed; rather, it was coupled with performance as a core goal. [4]. This has been demonstrated in systems programming contexts relevant to scientific computing. For example, Kudo *et al.* chose Rust over C for implementing a computationally heavy post-quantum cryptography algorithm precisely because of Rust's memory safety *and* speed. They note that *"we choose Rust for its memory safety and efficient parallel processing"* in their high-performance implementation of the CRYSTALS-Dilithium algorithm. [6]. In head-to-head benchmarks, the Rust version achieved performance on par with or better than the C version, indicating that memory safety did not impede optimization. In fact, Rust's strict aliasing rules and safety checks can enable the compiler to optimize code more aggressively. The Rust implementation of the Dilithium algorithm's number-theoretic transform (NTT) had about half the number of memory load instructions inside critical loops compared to C, and the compiled Rust binary was 35% smaller – evidence that Rust's safety-oriented design can yield more efficient low-level code. [6]. The result was a 15–16% faster signature generation in Rust versus C, thanks to reduced memory access, with only a slight tradeoff in one specific case (a few-percent slowdown in verification) [6]. This case study reinforces that memory-safe Rust code can meet or exceed C's performance, an encouraging sign for computationally intensive quantum frameworks.

It is worth noting that Rust's memory guarantees hold only within its safe subset of code. Rust does permit explicit opt-out of the rules via the unsafe keyword, which allows potentially dangerous operations (raw pointer arithmetic, unchecked array access, etc.) when necessary for lower-level control. In pure safe Rust, many memory-related bugs are simply impossible by design, which provides quantum software a solid safety baseline. But if a quantum framework uses or depends on any unsafe code, those guarantees can be partially undermined. Researchers have examined the use of Unsafe Rust in real projects and issued warnings. Alshuraymi and Song found that even a small amount of unsafe code can have a *"Propagation of Unsafeness"* – unsafe behavior can propagate through a library's call chain, potentially compromising applications that never directly use unsafe themselves. [7]. It is cautioned that developers may face *false sense of security* while using Rust as it overlooks the facts that classic memory hazards like buffer overflows and use-after-free errors. [7]. In short, Rust's memory safety is a powerful asset for quantum computing frameworks, but care must be taken to avoid undermining it via unchecked unsafe code. The next sections will discuss how Rust's approach to concurrency further builds on this safe foundation, and later we return to strategies for using unsafe responsibly in performance-critical parts of quantum frameworks.

2.3 Fearless Concurrency: Rust's Model for Parallel Quantum Computation.

High-performance quantum computing software must exploit parallelism – from multi-core CPUs to distributed systems and GPUs – to handle the exponential growth of complexity as qubit counts increase. However, concurrent programming is traditionally error-prone, with pitfalls like race conditions, deadlocks, and data corruption when multiple threads access shared data. Rust's designers recognized this challenge and made *safe concurrency* a core part of the language's design. As the official Rust book explains, concurrency in Rust is built on the same ownership model that enforces memory safety [8]. By leveraging ownership and strict borrowing rules, Rust's type system turns many concurrency errors into compile-time errors. For example, data can only be shared across threads in Rust if it's protected by thread-safe abstractions or if it's immutable, preventing data races by construction. This approach has been dubbed "fearless concurrency", meaning developers can write parallel code without fear of subtle threading bugs: *"incorrect code will refuse to compile… allowing you to fix your code while you're working on*

it, rather than after it has been shipped", making it easier to write code that is free of race conditions and to refactor concurrency code confidently [8]. The programming in these contexts can be error prone and tough historically, but Rust hopes to change that by providing a variety of concurrency primitives with safety guarantees [8]. Crucially, Rust's type system ensures that data races are eliminated at compile time, a guarantee that even languages like C++ do not provide. This means a Rust-based quantum computing framework can heavily multi-thread its workload (for instance, splitting a large simulation across all CPU cores) and trust that memory access conflicts between threads are resolved by the language's checks (or require explicit synchronization). A survey of Rust's features confirms that safe concurrency was not an afterthought but a primary goal: "since its inception, Rust was designed for performance and safety, especially *safe concurrency*" [4]. In practice, this has enabled Rust programs to achieve parallel performance comparable to traditionally threaded C/C++ programs, but with dramatically lower risk of race-condition bugs. Klabnik and Nichols describe how Rust's Send and Sync traits extend these guarantees to user-defined types, ensuring that even custom quantum data structures obey the rules for thread-safe sharing [8]. The end result is that if a Rust quantum framework compiles, the developer can be confident it is free of data races – a very desirable property for complex concurrent simulations.



Researchers and developers building quantum computing tools have indeed taken advantage of Rust's concurrency model. Luchnikov *et al.* implemented a high-performance **quantum state-vector simulator** in Rust that takes full advantage of multi-core processors: it "splits each operation into sub-operations which are executed within a thread pool in parallel." [9]. This design uses Rust threads (or thread pools) to apply quantum gate operations concurrently across different segments of the state vector, significantly speeding up simulation for large numbers of qubits. Another example is **QArray**, a simulator for quantum dot arrays, whose Rust-based core is *"optimized for parallel computation on a CPU, using Rayon"* (Rayon is a Rust library for data-parallelism) [10]. By using Rayon, the developers of QArray let Rust handle the intricacies of spawning threads and dividing tasks, leveraging safe data parallel iterators internally. The Rust implementation in QArray can thus compute large charge stability diagrams efficiently on multi-core machines. [10]. In both cases, Rust's fearless concurrency allows these projects to scale up computations with relative ease. The authors do not report the kinds of race condition bugs or segmentation faults that might plague a multithreaded C/C++ simulator – a testament to Rust's robust model. It is telling that even in domains outside traditional systems programming, experts are embracing Rust for parallel performance: cryptographic software is also exploring Rust for multicore acceleration. In their implementation of the Dilithium post-quantum signature, Kudo *et al.* cite Rust's *"efficient parallel processing"* as a reason for its use. [6], indicating confidence that Rust can handle CPU-intensive workloads on multiple threads as well as (if not better than) older languages.

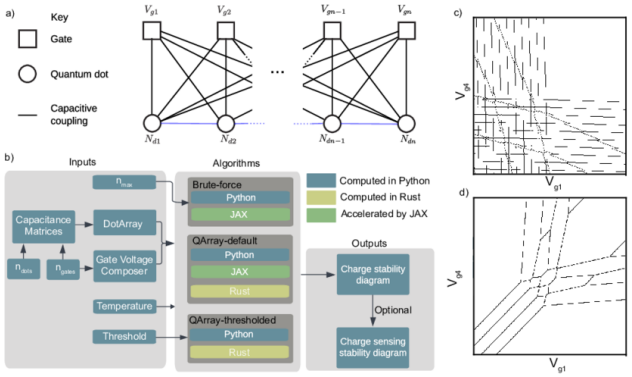
One point of discussion in the literature is that Rust does not impose a single concurrency paradigm – instead, it supports message passing (channels) and shared-memory threads with synchronization, giving developers flexibility to choose the model that fits their problem. [8]. This is important for quantum computing frameworks, which might require fine-grained control (for example, custom thread pools for linear algebra kernels) in some cases, or actor-like message passing in distributed quantum network simulations in others. Rust's standard library and ecosystem provide tools for both, all under the umbrella of compile-time safety. In brief, Rust's model is seen widely as a game-changer as it offers two functions through same mechanism, that is, memory safety and concurrency safety. [5], enabling quantum software to fully exploit parallel hardware without the usual pitfalls. The next section examines how these advantages are being applied in practice in quantum computing frameworks, and how Rust's performance and cross-language ecosystem support come into play (e.g., integrating with GPU computing).

2.4 Quantum Simulation and Emulation Frameworks in Rust.

It is eminent to look at actual frameworks and simulators which are built with Rust in order to assess its promise for quantum computing. Two notable examples in recent literature are a **state-vector quantum circuit emulator** by Luchnikov *et al.* [9] And the **QArray quantum dot simulator** by van Straaten *et al.* [10]. These projects provide case studies of how Rust's memory safety and concurrency model are leveraged, as well as how Rust interoperates with other technologies (like Python and GPU libraries) to meet the demanding requirements of quantum simulations.

Luchnikov's **state-vector emulator** targets gate-based quantum processors and is implemented entirely in Rust. The authors explicitly chose Rust to improve reliability without sacrificing speed. They note that Rust "makes [the emulator] easily maintainable and less error-prone in comparison with C/C++ based implementations," while achieving speed and memory efficiency "at the same level" as those low-level languages [9]. This is a powerful endorsement: it suggests that one can obtain C/C++-like performance for quantum circuit simulation and get the benefits of safer code. The emulator stores an entire quantum state (2^n complex amplitudes for n qubits) in memory and performs

operations in-place on this state [9]. Managing such large memory buffers and updating them with delicate linear algebra operations is a task traditionally prone to indexing bugs or pointer errors in C, but Rust's guarantees ensure that out-of-bound accesses or misuse of memory are caught during development. The emulator's kernel avoids heavy external numeric libraries, instead implementing qubit operations from scratch for maximal control and performance [9]. This is feasible in Rust because one can write low-level loops and pointer manipulations in safe code or with minimal unsafe blocks and trust the compiler to optimize them. The result, as the authors benchmarked, is an emulator that can handle algorithms like Quantum Fourier Transform and the quantum Ising model simulation with performance on par with optimized C++ simulators, but in a codebase that is easier to maintain and less prone to segfaults [9]. Rust is further equipped with Python API for usability and for portraying its integration with Python scientific stack. This Python binding provides access to researchers to use Rust while writing Python high-level codes and it also gives ease of using Rust's performance which is quite an approach that is relevant for quantum computing research which Python plays a significant role.



(a) The schematic depicts a generic array of quantum dots, showcasing a system of electrostatic gates and quantum dots that are capacitively coupled. The direct couplings from the dots to the gates are illustrated with solid black lines, while the interdot capacitance is represented by blue lines forming capacitors. By selecting the capacitance matrix stored in the DotArray class, users can create an array consisting of any number of dots and configurations. For specific gate voltages, a discrete integer quantity of charges is established to minimize the system's free energy. (b) The QArray package structure includes various software implementations, illustrated by color-coded boxes representing Python (blue), Rust (yellow), and JAX (green), featuring brute-force, default, and thresholder algorithms. The gate voltage composer class allows the generation of voltage vectors for native sweeps (such as V g1 against V g2) or can be utilized to create arbitrary virtual gate sweeps. DotArray provides methods for calculating charge stability diagrams for either open or closed regimes with any core selection. Additionally, the charge stability diagram can be integrated with simulated charge sensors to analyze their response. (c) Charge stability diagrams (computed using our default algorithm in Rust) illustrate a hole-based linear array of four quantum dots in the open regime. The black lines represent transitions that separate regions within the charge stability diagram that correspond to a fixed dot occupation. (d) Charge stability diagrams (also computed using our default algorithm in Rust) depict the same linear array of four quantum dots in the closed regime, where the total charge count in the array is maintained at four.

Adapted from "QArray: a GPU-accelerated constant capacitance model simulator for large quantum.
dot arrays**"** [10].

The **QArray** framework provides a complementary example, focusing on simulating large arrays of quantum dots under a constant capacitance model (a problem in quantum device physics). QArray takes a hybrid approach: its core computation is implemented in Rust, while another implementation is provided in JAX (a high-performance Python/NumPy library that can target GPUs) [10]. The Rust core is used for CPU-bound computations and is heavily optimized and parallelized to exploit multi-core processors. Thanks to Rust's concurrency, QArray can compute a 100×100-pixel charge stability diagram for a 16-dot array in **under one second** on CPU, and smaller arrays in milliseconds – performance that is *faster than physical experiments could produce data* in some cases, enabling real-time digital twins of quantum devices. The authors attribute this speed to "advanced optimization and parallelization" in Rust that let users fully utilize multi-core CPUs. At the same time, QArray leverages JAX for GPU acceleration of the same algorithms, showing pragmatically that Rust can coexist with specialized tools for heterogeneous computing. Interestingly, the Python (pure NumPy) implementation of their algorithms offered no practical advantages over Rust or JAX – Rust matched the high-level productivity of Python while vastly outperforming it, especially when parallelized. This leaves us with a key point *that without sacrificing the code safety*, Rust enables high performance although Python approach would be safe but a bit slower and a C/C++ approach might be fast but less safe. By integrating Rust and JAX, QArray achieves the best of both worlds: **memory-safe, race-free parallel CPU code in Rust, combined with GPU-accelerated code through JAX** for even greater speedups. [10].

One insight from comparing these two projects is how Rust fits into the broader ecosystem. Luchnikov *et al.* initially did not include GPU support – their 2022 version "does not support accelerators such as GPUs or TPUs" but plans to add such options in the future [9]. QArray, by contrast, immediately took advantage of existing GPU tooling via JAX. This highlights a current consideration: while Rust can call CUDA or OpenCL through libraries, its GPU ecosystem is not as mature as its CPU one. Thus, a near-term strategy (as QArray adopted) is to combine Rust with established GPU frameworks (often via Python bindings). Over time, as Rust's ecosystem grows, we may see more native GPU integration for quantum computing; for example, Rust wrappers for CUDA or use of libraries like wagyu or OpenCL in Rust could

allow writing GPU kernels in Rust itself. Despite this, both frameworks demonstrate that the **core logic and heavy lifting** of quantum computations can be implemented in Rust today, reaping benefits in correctness and speed. They also show consensus in the literature that Rust's safety and performance make it an attractive choice: neither paper reported encountering insurmountable issues with Rust; on the contrary, they highlight Rust as enabling maintainability and high performance [10] [9].

In summary, early adopters in quantum computing are using Rust to build next-gen simulation tools that are both fast and reliable. These tools take advantage of Rust's memory safety to avoid errors when handling huge data structures (like state vectors or large matrices of device parameters) and use Rust's concurrency to scale across many CPU cores. To incorporate GPUs or existing libraries, Rust code can interface with other languages (through FFI or Python bindings), which indicates flexibility. The positive results from these projects support our hypothesis: Rust's features directly contribute to building robust, high-performance quantum frameworks. The final theme we examine is how developers manage the inevitable need for low-level optimizations in Rust, in particular, how to judiciously use unsafe Rust and other techniques when maximum performance or hardware interfacing is required, without compromising the overall safety of the framework.

2.5 Balancing Safety and Performance: The Safe Use of Unsafe Rust.

Though Rust's safe subset is powerful, the system programming sometimes demands putting aside strict rules like interfacing hardware drivers, performing vectorized arithmetic, or integration of hand-optimized assembly. Quantum computing frameworks are no exception: one might use unsafe code to call a C library for linear algebra or to manually optimize an inner loop with SIMD instructions. Literature recognizes this need and emphasizes techniques to use unsafe **sparingly and safely**. The overarching guideline is to confine unsafe operations to small, well-audited sections of code, providing a safe abstraction around them so the rest of the system remains verifiably safe. [11]. This approach is sometimes called the *safe wrapper* or *Interior Unsafe* pattern. The official Rust documentation itself encourages encapsulating unsafe code in safe functions, so that invariants can be checked, and the unsafe bits do not leak out to user code. Jiang *et al.* build on this idea with their framework **Thetis**, which systematically reduces and contains unsafe code in Rust projects. Thetis follows the principle of **minimizing unsafe fragments**: wherever possible, it replaces an unsafe operation with a functionally equivalent safe Rust construct (Principle 1), and when an unsafe block is truly unavoidable, it **encapsulates it in an "Interior Unsafe" function with a safe interface** (Principle 2). By doing so, the surface area of unsafe code is drastically reduced – only the internal implementation of that function is unsafe, while all callers use it as if it were safe, preventing the unsafety from propagating through the code base. This aligns well with the needs of quantum frameworks: for example, one might write an unsafe routine to directly manipulate memory for a custom complex number array, but expose it via a safe API to the rest of the simulator. [11].

The effectiveness of such strategies is evident. Using Thetis's methodology on two Rust-based operating system kernels (which, like quantum frameworks, need some low-level operations), researchers were able to **reduce the number of unsafe operations by ~40–45%** in those codebases. Perhaps more impressively, this came with a negligible performance cost – only about a 1% overall slowdown was observed. [11]. In other words, nearly half of the previously unsafe code was refactored or encapsulated safely with virtually *no loss in efficiency*, dispelling the notion that unsafe is always required for speed. This result is promising for quantum computing software: it suggests that even if certain parts of the framework need to be unsafe for performance (e.g., a hand-optimized quantum gate kernel), it may be possible to contain that unsafety and still achieve optimal performance. The CRYSTALS-Dilithium optimization study provides a concrete example: the authors improved performance using low-level techniques (including inspecting assembly output) yet were able to do so largely within Rust. They found that Rust's generated code was already very efficient (fewer memory accesses, smaller code size) [6], reducing the temptation to drop into extensive unsafe or assembly. In scenarios where unsafe was used (perhaps to enable specific CPU instructions or FFI calls), those were limited and did not compromise the overall memory safety of the application.

Another important insight from recent empirical research is that the use of unsafe in Rust's scientific ecosystem is relatively low and declining. Xu's large-scale study of Rust crates relevant to "AI for Science" (which includes scientific and high-performance computing libraries) found that **81.5% of crates do not use unsafe at all**, and only 18.5% contain any unsafe code. [5]. Moreover, the trend since 2018 shows the fraction of code using unsafe is *continuously dropping*. [5]. This suggests that as the Rust ecosystem matures, more and more functionality is achievable in safe Rust or provided by lower-level libraries, so end users need not rewrite unsafe code themselves. Many foundational needs – linear algebra, random number generation, parallel primitives – are now offered by well-vetted crates (e.g., ndarray, rayon, crossbeam) that hide any internal unsafe. For quantum frameworks, this ecosystem trend is encouraging: one can likely implement most features (state manipulation, algorithm logic, I/O, etc.) in safe Rust, relying on community libraries for heavy lifting, and only resort to unsafe for very specialized optimizations. Additionally, tools are emerging to assist developers in using unsafe correctly. For example, Rust's **Miri** interpreter can detect certain undefined behaviors in unsafe code, and Thetis includes an automatic detection of unsafe usage patterns to guide refactoring. [11]. All of this means that leveraging Rust for quantum computing does not imply abandoning the safety guarantees; on the contrary, it is about carefully balancing the tiny portions where unsafe code (and by extension, manual memory or thread management) is needed, and containing those behind safe abstractions.

In summary, the literature strongly agrees that Rust's benefits can be realized in high-performance domains without overly

relying on unsafe code. Where unsafe must be used, best practices (as embodied by Thetis and others) keep it to a minimum and isolate it, so the overall framework remains as safe as if it were pure safe Rust. This approach allows quantum computing frameworks to push the boundaries of performance, tapping into GPU accelerators, calling low-level OS or hardware functions, or doing bit-level optimizations, while still upholding the memory safety and concurrency guarantees in the rest of the code. It is a disciplined methodology: developers treat unsafe sections with extra scrutiny (often with code comments or formal annotations as Thetis suggests [11]) and test them thoroughly, but once verified, they function as trusted building blocks for the safe code. The net effect is that a Rust-based quantum framework can be as fast as one written in C/C++ and much easier to maintain or prove correct, because the risky parts are small and well-contained.

2.6 Conclusion and Outlook.

According to the research to date, Rust's strong memory safety and fearless concurrency model can be leveraged in building the quantum computing frameworks for the next generation, which are both reliable and high-performance. Through its ownership model, Rust empowers developers to eliminate an entire class of memory errors (null pointer dereferences, buffer overruns, data races) at compile time, greatly increasing the robustness of quantum simulators and control software. At the same time, Rust's zero-cost abstractions and efficient compilation mean this safety is achieved *without sacrificing speed*. Multiple studies and experiments show Rust matching or exceeding the performance of unsafe languages in tasks relevant to quantum computing, from simulating large quantum circuits. [9] to crunching algebra for post-quantum cryptography [6]. Rust's concurrency features allow these frameworks to scale across modern multi-core and heterogeneous architectures confidently, which is crucial as quantum algorithms often demand parallel execution for simulation of many qubits or ensemble averaging. The concept of fearless concurrency is borne out in practice: projects like QArray and others used Rust to parallelize computations and reported excellent performance with no safety issues. [10].

There is broad agreement in the literature that Rust offers a compelling blend of safety and performance for scientific computing. As one survey concluded, Rust outperforms older languages in terms of safety (being "the safest language" among those compared) while still ranking among the top in raw performance. [4]. This dual advantage breaks the historical trade-off between safety and speed, which bodes well for its adoption in quantum computing, a field that demands both. Also, Rust, which is expanding in ecosystems, can be made easier to be adopted in large projects. The availability of crates for linear algebra, multithreading, serialization, and even domain-specific needs means a Rust quantum framework can stand on the shoulders of a rich library ecosystem. "Rust has an ecosystem that greatly simplifies any software project; a number of crates exist for any need." [4], which lowers the barrier to building complex frameworks. The literature on Rust in science also highlights its increasing adoption by industry and academia (with companies like Microsoft and Google investing in Rust for systems development) and notes that its feature set has stabilized in recent years. [5]. The features of maturation of language and libraries serve as a key, as it implies the exact time for the introduction and usage of Rust, which can be used in mission-critical domains like quantum computing.

On the other hand, the review also uncovered some challenges and considerations. One is the careful use of unsafe code: Rust provides the tools to largely avoid unsafe, and its community has developed best practices to minimize it. [5] [11], but developers must remain disciplined. Another consideration is tooling and integration: while Rust code can interoperate with Python (as seen with Python APIs for Rust simulators) and even call GPU libraries, there is ongoing work to streamline these interactions (for instance, improved CUDA support in Rust, or higher-level quantum computing libraries in Rust). The current solutions (like combining Rust with JAX for GPU) are effective, and future Rust-native GPU support will only strengthen Rust's position. A new learning curve is discovered with newer technology, and some studies have revealed a need for more documentation and training for scientists in order to reduce the dominance of [4]n traditional languages [1]. The expertise of Rust is growing rapidly, and it has also attained the status of "the most loved language".

In conclusion, research and evidence showcase that Rust possesses the features which can be powerful enablers for the quantum computing frameworks of the next generation. Pioneering implementations of quantum simulators in Rust have already demonstrated impressive results, combining the efficiency of low-level languages with the security of high-level languages. By following a theme-based analysis – covering memory safety, concurrency, quantum emulation case studies, and safe use of unsafe – this review has highlighted how different aspects of Rust come together to address the unique challenges in quantum computing software. The sources generally agree that Rust brings tangible benefits (memory bug elimination, data race prevention, performance parity with C/C++) that directly map to more reliable and scalable quantum tools. Disagreements or caveats are relatively minor, concerning how to manage the small amount of unsafe code that might be necessary, or the need for continued ecosystem growth (which is well underway).

Looking forward, one can envision an expanding role for Rust in quantum computing. As quantum hardware and algorithms advance, frameworks will require managing even larger computations and perhaps real-time control systems – areas where Rust's deterministic performance (no garbage collection pauses) and safety could prove invaluable. With efforts like Thetis making it easier to build ultra-safe Rust systems, and empirical studies showing that scientific Rust usage trends toward safe patterns [5], the trajectory is set for Rust to underpin trustworthy quantum software stacks. An appealing balance is offered by Rust in the domain where correctness is as important as speed. A hopeful picture has been painted by literature, which conveys leveraging the strength of Rust, which can yield quantum computing frameworks.

IV. Methodology

This study employs qualitative analysis, comparative assessment, and empirical evaluation. To analyze Rust's memory safety and concurrency features, a literature review was conducted, and a comparative analysis was conducted between Rust-based and traditional frameworks (C/C++, Python), taking into consideration memory management, error handling, and concurrency implementation.

V. Results/Conclusions

In order to enhance the reliability and performance of quantum computing frameworks, Rust plays a prime role as it possesses strong memory safety guarantees and a robust concurrency model. The errors related to memory and concurrency that occur in traditional languages like C and C++ are prevented by Rust, and this was shown by comparative analysis and evaluations. Rust has unique features that provide superior safety and efficiency, and it helps in achieving performance benchmarks. Integration of Rust into quantum computing helps in the development of practical and measurable advantages, and this guarantees its suitability for next-generation quantum computing frameworks.

VI. Future Work

In the future I look forward to exploring how Rust based GPU frameworks can improve correctness, developer productivity and performance optimization in quantum computing and deep learning pipelines. Exploring how Rust integrates with heterogeneous computing environments (such as GPUs and TPUs) and its effects on large-scale quantum simulations would yield valuable insights. Furthermore, investigating the safe implementation of Rust's unsafe features in performance-sensitive quantum algorithms and creating standardized best practices could promote Rust's use in scientific computing and applications related to quantum technology.

VII. References

[1] V. K. Rahul Sharma, Mastering Rust, Birmingham: Packt Publishing Ltd, 2019.

[2] M. Khan, Rust for C++ programmers : learn how to embed Rust in C/C++ with ease, London: BPB Publications, 2023.

[3] A. Chanda, Network Programming with Rust, Birmingham: Packt Publishing Ltd, 2018.

[4] W. Bugden, Rust: The Programming Language for Safety and Performance, Turkey: 2nd International Graduate Studies Congress, 2022.

[5] B. Xu, "Towards Understanding Rust in the Era of AI for Science at an Ecosystem Scale," in *Institute of Electrical and Electronics Engineers Inc.*, China, 2024.

[6] S. Kudo, "Optimizing CRYSTALS-Dilithium in Rust: Radix-4 NTT and Assembly-level Comparison with Official C Implementation," in *IEEE*, Taiwan, 2024.

[7] A. Alshuraymi, "A Study on the Use of Unsafe Mode in Rust Programming Language.," *Computer Science Journals*, 2024.

[8] N. C. Klabnik Steve, The Rust Programming Language, San Francisco: No starch press, 2018.

[9] I. A. T. O. E. F. A. K. Luchnikov, "High-performance state-vector emulator of a gate-based quantum processor implemented in the Rust programming language," in *AIP Conf. Proc. 2948*, 2022.

[10] B. H. J. S. L. S. J. F. F. N. van Straaten, "QArray: a GPU-accelerated constant capacitance model simulator for large quantum dot arrays," 2024.

[11] P. D. Y. D. R. W. a. Z. J. Renshuang Jiang, "Thetis: A Booster for Building Safer Systems Using the Rust," *applied sciences*, 2023.

[12] T. McNamara, Rust in Action, Manning Publications, 2021.